PHP

Desenvolvimento Web com PHP

Fundamentos e Integração

Variáveis, Lógica e Comunicação com o Navegador

Programação Web II • Ronildo | Madureira

Variáveis e Tipos de Dados

📌 **Conceito:** Espaços na memória que armazenam dados durante a execução do programa.

🔷 **Sintaxe:** Sempre precedidas pelo símbolo **$**

📦 **Tipos Principais (Exemplos Práticos):**

Integer	`$idade = 25;`
Float	`$altura = 1.75;  ← IMC!`
String	`$nome = "Google Slides";`
Boolean	`$logado = true;`

⚡ **Type Casting:** `$valor = (float)"10.5";` — *Força a conversão de tipo*

Estruturas de Decisão

🎯 Objetivo: Alterar o fluxo do código com base em condições verdadeiras ou falsas.

if / else
Executa blocos baseados em verdadeiro ou falso.

switch
Melhora a leitura com muitas opções para uma mesma variável.

⚙️ **Fluxo de Decisão**

Condição?

✅ Verdadeiro → Executa bloco if

❌ Falso → Executa bloco else

💻 **Exemplo (de imc.php):**

imc.php

```php
if ($imc < 18.5) {
    echo "MAGRO";
} else if ($imc < 30) {
    echo "NORMAL";
}
```

Laço FOR — Quando sabemos quantas vezes repetir

1 de 3

Ideal quando o número de repetições é conhecido antes de entrar no laço.

🎯 **Problema: Contar de 1 até 5, exibindo cada número na tela**

🔬 **Anatomia:**

①	`$i = 1`	Inicialização — executa uma vez
②	`$i <= 5`	Condição — checa antes de cada volta
③	`$i++`	Incremento — executa após cada volta

💻 **Solução com FOR:**

contadorFOR.php

```php
for($i = 1; $i <= 5; $i++) {
  echo "Contagem: " . $i . "\n";
}

// $i começa em 1
```

📟 **Saída:** `Contagem: 1` `Contagem: 2` `Contagem: 3` `Contagem: 4` `Contagem: 5`

Laço WHILE — Quando a condição é verificada antes

2 de 3

Usado quando não sabemos de antemão quantas vezes repetir — checa a condição ANTES de entrar.

🎯 **Mesmo problema: Contar de 1 até 5 — agora com WHILE**

🔄 **Fluxo do WHILE:**

1. Inicializa $i FORA do laço
2. Avalia a condição ($i <= 5)
3. ✅ **Verdadeiro → executa o bloco**
4. Incrementa $i++ dentro do bloco

💻 **Solução com WHILE:**

contadorWHILE.php

```php
$i = 1;  // inicializa ANTES

while ($i <= 5) {
  echo "Contagem: " . $i . "\n";
  $i++;  // incrementa DENTRO
```

📟 **Saída:** Contagem: 1 Contagem: 2 Contagem: 3 Contagem: 4 Contagem: 5 ← **idêntica ao FOR!**

Laço DO-WHILE — Executa Pelo Menos Uma Vez

3 de 3

A grande diferença: verifica a condição DEPOIS — o bloco sempre executa ao menos uma vez.

🎯 **Mesmo problema: Contar de 1 até 5 — agora com DO-WHILE**

⚡ **Comparativo: WHILE vs DO-WHILE com condição falsa desde o início ($i = 99):**

WHILE (condição ANTES)

```
while                    // $i=99
```

→ bloco NUNCA executa ❌

VS

DO-WHILE (condição DEPOIS)

```
do        while          // $i=99
```

→ bloco executa 1 vez ✅

💻 **Solução com DO-WHILE:**

contadorDOWHILE.php

```
$i = 1;
do {
  echo "Contagem: " . $i . "\n";
  $i++;
} while ($i <= 5);
```

Saída dos 3 laços — idêntica!

```
Contagem: 1
Contagem: 2
Contagem: 3
Contagem: 4
Contagem: 5
```

Funções e Procedimentos

🔵 **Função**
Bloco nomeado que processa dados e retorna um valor usando return.

🟣 **Procedimento**
Apenas executa uma tarefa sem devolver dados ao chamador.

✅ **Vantagem**
Evita repetição de código — favorece o reuso e a manutenção.

💻 **Exemplo (de funcfatorial.php):**

funcfatorial.php

```php
function calcFatorial($nfatorial) {
  $calculo = 1;
  for($i=2; $i<=$nfatorial; $i++)
  {
    $calculo = $calculo * $i;
  }
  return $calculo;
}
```

💡 **Dica:** Funções com `return` devolvem um valor; sem ele, são apenas procedimentos.

Formulários HTML (Interface)

🎯 `action`	Para qual arquivo PHP os dados serão enviados após o submit.
📤 `method`	Como os dados viajam: POST (escondido) ou GET (pela URL).
🏷️ `name`	Etiqueta que o PHP usa para identificar e capturar o campo.

💻 **Exemplo real — fatorial.html:**

fatorial.html

```html
<form action="fatorial.php"
      method="POST">
  <input type="text"
      name="txtFatorial"/>
  <input type="submit"
      value="Calcular"/>
</form>
```

📌 **Atenção:** O atributo `name` no HTML é a chave que o PHP usa nos superglobais para capturar o valor.

Captura de Dados no PHP — Superglobais

O PHP utiliza Superglobais para ler o que foi enviado pelo formulário:

`$_POST["campo"]`	Captura dados enviados pelo corpo da requisição (method POST). Ideal para senhas e dados sensíveis.
`$_GET["campo"]`	Captura dados enviados pela QueryString (URL). Visível no endereço do navegador.
`$_REQUEST["campo"]`	Captura dados de AMBOS os métodos.

```
// Exemplo: $num = $_POST["txtFatorial"];
```

Recebendo Dados na Prática — Exemplo Fatorial Completo

① fatorial.html
Usuário digita o número e clica em Calcular

→

② fatorial.php
Captura via $_REQUEST e chama a função

→

③ funcfatorial.php
Calcula com FOR e retorna o resultado

fatorial.html

fatorial.html

```html
<form
 action="fatorial.php"
 method="POST">

 Numero:
 <input
  type="text"
  name="txtFatorial"/>

 <input
  type="submit"
  value="Calcular"/>
</form>
```

fatorial.php

fatorial.php

```php
require_once
 ("funcfatorial.php");

// captura o valor
$fat = $_REQUEST
 ["txtFatorial"];

// chama a função
$result =
  calcFatorial($fat);

echo "Fatorial: "
  . $result;
```

funcfatorial.php

funcfatorial.php

```php
function
 calcFatorial(
  $nfatorial) {
  $calculo = 1;

  for($i=2;
    $i<=$nfatorial;
    $i++) {
    $calculo*=$i;
  }
  return $calculo;
}
```

Usuário digita **5** → PHP calcula 5! = 5×4×3×2×1 → Exibe: "Fatorial do numero digitado e: 120"

Métodos HTTP e URL

GET

🔓 **Dados visíveis na URL**

Os dados aparecem após o símbolo **?** (QueryString).

Exemplo:

```
pagina.php?id=5&nome=teste
```

✅ **Bom para:** buscas, filtros, bookmarks.

VS

POST

🔒 **Dados no corpo da requisição**

Os dados viajam **"escondidos"** e não aparecem na URL.

✅ **Mais seguro para:**

- Senhas e dados sensíveis
- Grandes volumes de texto
- Formulários de cadastro

Não aparecem na barra de endereço!